

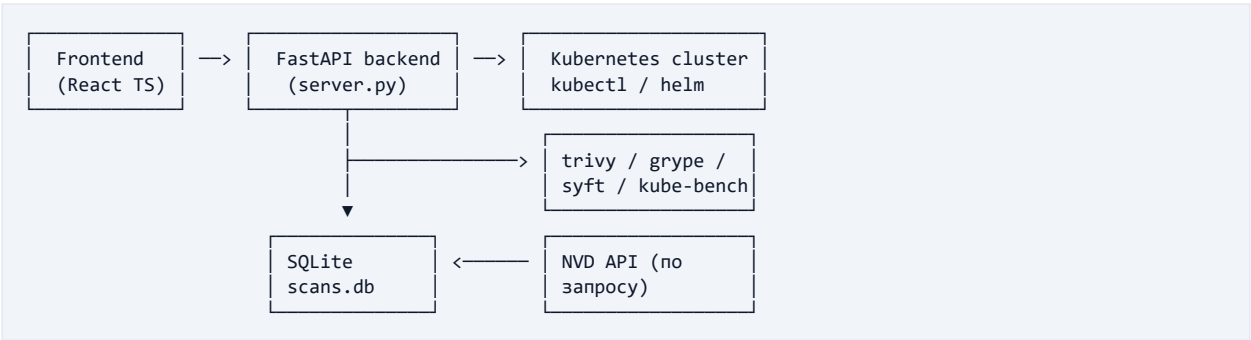
Security Lab

Система сканирования безопасности Kubernetes-кластера и контейнерных образов. Находит уязвимости (CVE), мисконфигурации ресурсов и нарушения CIS-бенчмарка, складывает результаты в единую базу и показывает их в веб-интерфейсе.

Содержание

- [Архитектура](#архитектура)
- [Возможности](#возможности)
- [Структура репозитория](#структура-репозитория)
- [Требования](#требования)
- [Развёртывание кластера (Ansible)](#развёртывание-кластера-ansible)
- [Запуск бэкенда](#запуск-бэкенда)
- [Запуск фронтенда](#запуск-фронтенда)
- [Переменные окружения](#переменные-окружения)
- [Сценарии работы](#сценарии-работы)
- [Справочник API](#справочник-api)
- [Категории сканов](#категории-сканов)
- [Модель данных](#модель-данных)
- [Траблшутинг](#траблшутинг)

Архитектура



- **Ansible** — разворачивает k8s-кластер и устанавливает сканеры на мастер-узел.
- **Backend (FastAPI)** — запускает сканеры, парсит вывод, хранит находки в SQLite, отдаёт REST API.
- **Frontend (React + TypeScript)** — UI для запуска сканов и просмотра результатов.

Возможности

Сканер	Что проверяет	Тип находок
Trivy (image)	CVE в образах helm-чарта	vuln

Сканер	Что проверяет	Тип находок
Grype (+ Syft SBOM)	CVE в образах (второе мнение) + генерация SBOM	vuln
Trivy k8s	Ресурсы namespace/кластера: образы + мисконфиги (privileged, RBAC, лимиты)	vuln, misconfig
kube-bench	Соответствие кластера CIS Kubernetes Benchmark	политики (FAIL/WARN)

Дополнительно:

- Обогащение CVE данными из **NVD** (описание, CVSS, ссылки, даты) — **по запросу**, не автоматически.
- Хранение и выгрузка **SBOM** (CycloneDX).
- Сравнение двух сканов (diff: что починили / что появилось).
- Разбивка сканов по категориям: образы / namespace / кластер / политики.

Структура репозитория

```
security-lab/
├── ansible/
│   ├── ansible.cfg
│   ├── inventory.txt
│   └── playbooks/
│       ├── 00-bootstrap.yaml      # базовая подготовка узлов
│       ├── 01-containerd.yaml     # установка containerd
│       ├── 02-kube-master.yaml    # kubeadm init на мастере
│       ├── 03-network.yaml        # CNI-сеть
│       ├── 04-kube-worker.yaml    # присоединение воркеров
│       ├── 05-trivy.yaml          # установка + скан Trivy
│       ├── 06-grype.yaml          # установка Syft + Grype, SBOM
│       └── 07-kube-bench.yaml      # установка + запуск kube-bench
├── server/
│   └── server.py                  # FastAPI backend (один файл)
├── security-lab-ui/              # React-фронтенд
│   └── src/
│       ├── api/scan.ts           # клиент REST API
│       ├── features/scans/       # страницы сканов
│       │   ├── cluster/         # namespace- и cluster-сканы
│       │   ├── ScanList.tsx     # сканы образов
│       │   └── ScanDetailsPage.tsx
│       │   └── ...
│       ├── features/vulnerabilities/
│       └── features/sbom/
└── charts/
    └── vuln-stack/               # тестовый helm-чарт с уязвимыми образами
```

Требования

На мастер-узле кластера (ставятся через Ansible или вручную):

- kubectl, helm
- trivy (≥ 0.40 — нужна подкоманда k8s)
- syft, grype
- kube-bench

Backend:

- Python 3.10+
- fastapi, uvicorn, python-multipart, pyyaml, pydantic

Frontend:

- **Node.js ≥ 18** (react 19 + react-scripts 5). На Node 12/14 dev-сервер не стартует.

Развёртывание кластера (Ansible)

1. Пропиши узлы в `ansible/inventory.txt` (группы `masters`, `workers`).
2. Прогоняй плейбуки по порядку:

```
cd ansible
ansible-playbook playbooks/00-bootstrap.yaml
ansible-playbook playbooks/01-containerd.yaml
ansible-playbook playbooks/02-kube-master.yaml
ansible-playbook playbooks/03-network.yaml
ansible-playbook playbooks/04-kube-worker.yaml
# сканеры:
ansible-playbook playbooks/05-trivy.yaml
ansible-playbook playbooks/06-grype.yaml
ansible-playbook playbooks/07-kube-bench.yaml
```

Запуск бэкенда

Бэкенд запускается **на мастер-узле** (нужен доступ к `kubectl/helm/сканерам`).

```
cd server
pip install fastapi uvicorn python-multipart pyyaml pydantic
uvicorn server:app --host 0.0.0.0 --port 8080
```

*! Порт **8080** — его ждёт фронтенд (`security-lab-ui/src/api/scan.ts`).*
SQLite-база создаётся автоматически в `server/data/scans.db`.

Запуск фронтенда

```
cd security-lab-ui
nvm use 20           # нужен Node >= 18
npm install
npm start            # дев-режим: http://localhost:3000
# или прод-сборка:
npm run build
```

Адрес бэкенда задан в `src/api/scan.ts` (`API_URL`). Если бэкенд на другом хосте/порту — поправь там.

Переменные окружения

Переменная	Где	Назначение
API_KEY	backend	Если задана — все запросы требуют заголовок X-API-Key. Пусто = авторизация выключена.
NVD_API_KEY	backend	Ключ NVD. С ключом лимит запросов выше (пауза 0.7с вместо 6с между CVE).
REACT_APP_API_KEY	frontend	Прокидывается в заголовок X-API-Key. Должен совпадать с API_KEY бэкенда.

```
export API_KEY="super-secret"
export NVD_API_KEY="xxxxxxxx-xxxx-xxxx-xxxxxxxxxxxx"
uvicorn server:app --host 0.0.0.0 --port 8080
```

Сценарии работы

1. Скан helm-чарта (образы)

UI: кнопка **Scan image** → имя release + .tgz чарта. Бэкенд деплоит чарт во временный namespace, собирает образы подов и сканирует Trivy + Grype.

```
curl -X POST http://<host>:8080/scan/start \
-F "release=my-app" -F "chart=@charts/vuln-stack-0.1.0.tgz"
```

Для теста готов чарт charts/vuln-stack с 5 заведомо уязвимыми образами.

2. Скан namespace (Trivy k8s)

UI: **Namespaces scan** → выбрать/вписать namespace → «Сканировать namespace». Результаты появляются на этой же странице (статус опрашивается автоматически).

```
curl -X POST "http://<host>:8080/scan/k8s?namespace=default"
```

3. Скан всего кластера

UI: **Cluster scan** → две кнопки: Trivy (весь кластер) и kube-bench (CIS).

```
curl -X POST "http://<host>:8080/scan/k8s" # trivy, весь кластер
curl -X POST "http://<host>:8080/scan/kube-bench" # CIS benchmark
```

4. Обогащение из NVD (вручную)

Автоматически в NVD ничего не ходит. Когда нужно — дёргаешь ручку:

```
curl -X POST http://<host>:8080/scan/<scan_id>/sync-nvd # по всему скану
curl -X POST http://<host>:8080/vulnerabilities/sync/nvd/CVE-2023-1234
```

NVD знает только CVE-, DLA-*, GHSA-* пропускаются.*
*Синк **дополняет** базу: заполняет пустые поля, уже записанные данные не перетирает.*
Признак, что CVE обогащён NVD — заполнены published_at / modified_at (их ставит только NVD).

Справочник API

Все сканы запускаются в фоне и сразу возвращают scan_id + status: running.
Статус и находки — через GET /scan/{scan_id}.

Сканы

Метод	Путь	Описание
POST	/scan/start	Скан helm-чарта (form: release, chart)
POST	/scan/k8s?namespace=	Trivy k8s; без namespace = весь кластер
POST	/scan/kube-bench	kube-bench (CIS)
GET	/scan/list?category=&page=&page_size=	Список сканов. category: image *(дефолт) / namespace / cluster / policy / all
GET	/scan/stats	Счётчики по статусам всех сканов (для дашборда)
GET	/scan/{scan_id}	Детали + находки (фильтры severity, scanner, q, пагинация)
GET	/scan/diff/{scan1}/{scan2}	Что починили / что появилось
DELETE	/scan/{scan_id}	Удалить скан (+ его namespace)
POST	/scan/{scan_id}/sync-nvd	Обогатить все CVE скана из NVD (фон)

SBOM

Метод	Путь	Описание
GET	/scan/{scan_id}/sbom?image=	Список SBOM скана
GET	/scan/{scan_id}/sbom/{sbom_id}/download	Скачать SBOM (CycloneDX JSON)

kube-bench

Метод	Путь	Описание
GET	/kube-bench/list	Список kube-bench-сканов со сводкой

Метод	Путь	Описание
GET	/kube-bench/{scan_id}	Детали kube-bench-скана

Уязвимости

Метод	Путь	Описание
GET	/vulnerabilities?severity=&q=	Каталог CVE
GET	/vulnerabilities/{cve_id}	Детали CVE
PATCH	/vulnerabilities/{cve_id}/description	Переопределить описание (body: {"description": "..."})
POST	/vulnerabilities/sync/nvd/{cve_id}	Синк одного CVE из NVD

Прочее

Метод	Путь	Описание
GET	/namespaces	Список namespace'ов кластера

Категории сканов

Тип скана определяется по полям `release` / `namespace` записи в таблице `scans`:

Категория	Условие	Где в UI
image	release ∈ {kube-bench, trivy-k8s}	страница «Сканы образов»
namespace	release=trivy-k8s и namespace ≠ all-namespaces	«Namespaces scan»
cluster	release=trivy-k8s и namespace = all-namespaces	«Cluster scan»
policy	release=kube-bench	«Cluster scan»

Модель данных

SQLite, файл `server/data/scans.db`.

- `scans` — `id`, `ts`, `namespace`, `release`, `status`.
- `scan_images` — образы, найденные в сканах helm-чартов.
- `scan_findings` — находки: `scanner`, `severity`, `target`, `cve_id`, `pkg_name`, `версии`, `cvss_score`, `description`, `finding_type` (`vuln/misconfig`).
- `vulnerabilities` — каталог CVE с обогащением из NVD/сканеров.
- `scan_sbom` — сохранённые SBOM (CycloneDX).

Траблшутинг

`npm start` **ничего не выводит / не стартует**

Проверь версию Node: `node -v`. Нужен ≥ 18 . На Node 12/14 `react-scripts 5` молча падает. Поставь Node 20 через `nvm` и переустанови `node_modules`.

`trivy k8s failed: unknown flag: --all-namespaces`

Устаревшая сборка бэкенда. Весь кластер сканируется без флага. Обнови `server.py`.

`node-collector-... already exists`

Прошлый `trivy k8s` оставил `job`. Бэкенд чистит его перед запуском; если завис прямо сейчас:

```
kubectl delete jobs,pods -n trivy-temp --all
```

Скан висит в running

Смотри логи бэкенда (там `print` из фоновых задач). Частые причины: нет `KUBECONFIG`/прав, недоступен сканер, `helm` не поднял поды.

NVD возвращает 403 / банит

Слишком частые запросы без ключа. Задай `NVD_API_KEY` — пауза снизится с 6с до 0.7с.

Фронт не достучался до API

Проверь `API_URL` в `src/api/scan.ts` (хост и порт **8080**) и совпадение `REACT_APP_API_KEY` с `API_KEY` бэкенда.